

Documentation report for Operating Systems Assignment 3

ATHANASIOS GOURDOMICHALIS

General Structure (multiproc1.c / multiproc2.c)

The general skeleton consists of the `main()` function with `int argc`, `char *argv[]` arguments. Based on the second argument (`argv[1]`), we extract the number of repetitions (`rep`), while the remaining arguments correspond to the strings assigned to each child process.

To prevent race conditions and avoid potential deadlocks, I used POSIX semaphores to enforce **mutual exclusion** among the processes. The implementation features two nested for loops: the outer loop iterates from $i = 0$ to `argc - 2` (one iteration per string / child process), while the inner loop invokes `display()` `rep` times for each respective process and string.

Furthermore, robust error handling is included: in cases of an invalid number of arguments, a negative `rep` count, or failures during `fork()` calls, an appropriate error message is displayed and the program exits gracefully.

I have implemented all required components of the assignment, and all submission test cases pass successfully. Finally, detailed inline comments have been added throughout the codebase for further clarification.

Issues Encountered During Implementation & Chosen Solutions. Below is a detailed breakdown of the challenges faced and the synchronization strategies implemented for both multiproc1.c and multiproc2.c:

1. **multiproc_1:** I did not encounter any particular issues with `multiproc1.c` or need to consider multiple alternative approaches, as my initial implementation passed 9/9 test cases on the first try.
 - **Argument Parsing & Validation:** The program begins by validating the command-line input, checking if `argc < 3`. It then converts the `rep` parameter to an integer using `atoi()` and ensures that `rep` is a positive integer. Appropriate error messages are printed if validation fails.
 - **Semaphore Setup:** I declared struct sembuf operations (`sem_op = -1` for down/wait and `sem_op = 1` for up/signal) to enforce mutual exclusion around the `display()` function, following the pattern shown in the lecture slides.
 - **IPC Resource Allocation:** Similarly to the slides, I allocated a single System V semaphore resource using `semget()` with `IPC_PRIVATE` and `0600` access permissions.

For Mutual Exclusion: When no more than one process is allowed to execute a critical region simultaneously:

1. Start with **sem = 1**
2. Entering the critical region: **DOWN(sem)**
3. Exiting the critical region: **UP(sem)**

- **Semaphore Initialization:** The semaphore value is initialized to 1 using `semctl()` with the `SETVAL` command to implement a binary semaphore for mutual exclusion.
 - **Child Process Creation:** A for loop iterates `argc - 2` times (once for each string), creating a child process via `fork()` in each iteration.
 - **Child Process Execution:** Inside the child process (`pid == 0`), an inner loop executes `rep` times to invoke the `display()` function. Each call to `display()` is enclosed by `down()` and `up()` operations on the semaphore to enforce mutual exclusion within the critical region. Once done, the child process exits gracefully.
 - **Child Tracking & Error Handling:** A counter keeps track of the total number of child processes successfully spawned by `fork()`, which is later used to govern the waiting loop. If `fork()` returns a negative value (`pid < 0`), execution continues to the next iteration.
 - **Parent Process Cleanup:** Finally, the parent process enters a loop to reap all created child processes using `waitpid()`. Once all children have terminated, the allocated semaphore set is removed from the system using `semctl()` with `IPC_RMID`.
2. **multiproc_2:** Building upon `multiproc1`, the key modification in `multiproc2.c` is the addition of a **shared counter** (`shared_counter`) stored in shared memory, accessible by all child processes. This counter tracks how many processes have executed `init()`, enabling inter-process synchronization after `init()` and before any process invokes `display()`. This guarantees that **no child process begins calling `display()` until all processes have completed their call to `init()`.**

- **Critical Region & Mutex:** The critical region here consists of calling `init()` and incrementing `shared_counter`. We enforce mutual exclusion over this critical section using the same binary semaphore interface as in `multiproc1`.

- **Barrier Synchronization & Spin Locking:** To synchronize all processes before `display()` -ensuring they wait for everyone to finish `init()` before executing `display()` concurrently-I utilized a **spin lock** approach:

```
while (*shared_counter < argc - 2) {  
    /* wait */  
}
```

This solution was inspired by the Producer-Consumer problem with a circular buffer from Discussion / Lab 3 slides. While the slides note that busy waiting "**wastes CPU time inside the while loop while the process holds the CPU**" this approach proved effective in practice: as soon as the last child process increments shared_counter to argc - 2, the while condition simultaneously evaluates to false across all waiting processes, achieving synchronization.

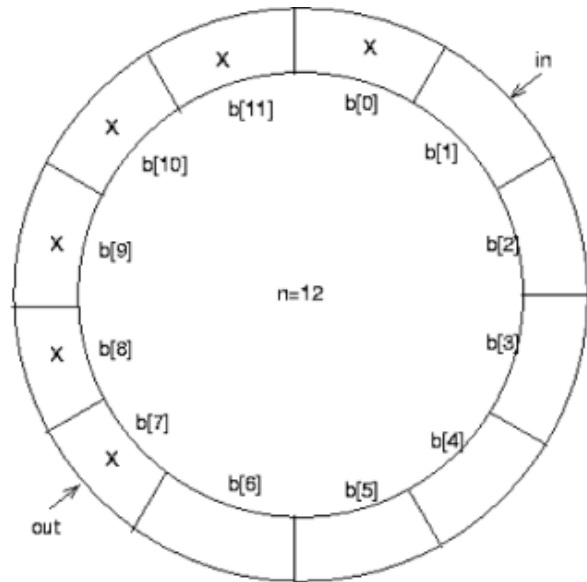
For reference:

```

• Producer:
1. while (true) {
2.     /*produce item v */
   while ((in + 1) % n ==
         out);
3.     b [in] = v;
4.     in = (in + 1) % n;
5. }

• Consumer:
1. while (true) {
2.     while (in == out);
3.     w = b [out];
4.     out = (out + 1) % n;
5. }

```



- **Shared Memory Lifecycle:** For shared memory management, I strictly followed standard System V IPC guidelines:
 1. Creation via **shmget()**
 2. Attaching via **shmat()**
 3. Detaching via **shmdt()**
 4. Destruction via **shmctl(shmid, IPC_RMID, NULL)** in the parent process.

Race conditions on the shared variable are carefully prevented by placing increments inside the semaphore-protected critical region. All other code logic remains identical to multiprocl.c.